Institute of Space Technology (KICSIT), Kahuta

Technical Project Report Transport Management System

Desktop Application & Containerized Deployment

Submitted By:

Jazib Asad

BS Computer Science

Section: 6th Semester

Submitted To:

Mr. Uzair Hassan

Department of CS

Development Environment:
Visual Studio 2022, MS SQL Server, Docker Desktop

Architecture:
Client-Server / Three-Tier / Containerized Infrastructure

April 2026

Contents

1 Abstract 2

2 System Architecture Analysis 2

2.1 Presentation Layer (UI) 2

2.2 Business Logic Layer (BLL) 2

2.3 Data Access Layer (DAL) 2

3 Database Design and Schema 2

3.1 Entity-Relationship Overview 2

4 Object-Oriented Programming (OOP) Application3

4.1 Encapsulation 3

4.2 Inheritance and Specialization 3

5 Containerization: Docker on Windows 3

5.1 The Dockerfile Process 3

5.2 General Deployment Commands 3

6 Security and Data Integrity 3

7 Conclusion 4

1 Abstract

The Transport Management System (TMS) is a specialized Enterprise Resource Planning (ERP) tool designed for the logistics sector. The project focuses on the digitalization of fleet operations, driver scheduling, and customer relationship management. By utilizing the .NET framework and ADO.NET for data persistence, the system provides a seamless interface for managing complex transport transactions, ensuring data integrity through relational database constraints and Object-Oriented Programming (OOP) principles.

2 System Architecture Analysis

The application utilizes a **Three-Tier (Layered) Architecture**. This separation of concerns ensures that the user interface is decoupled from the underlying data management, allowing for easier updates and debugging.

[Image of three-tier software architecture diagram]

2.1 Presentation Layer (UI)

The UI was developed using the C# Windows Forms framework. This layer is strictly responsible for capturing user input and displaying data. It utilizes event-driven programming to trigger interactions. Special attention was paid to UX/UI by incorporating custom icons and validated input fields.

2.2 Business Logic Layer (BLL)

The BLL acts as the ”brain” of the system. It handles calculations such as total trip costs and driver availability metrics before passing the data to the storage layer.

2.3 Data Access Layer (DAL)

The DAL manages the SqlConnection lifecycle. It utilizes SqlCommand and SqlDataAdapter to bridge the gap between the C# application and the SQL Server.

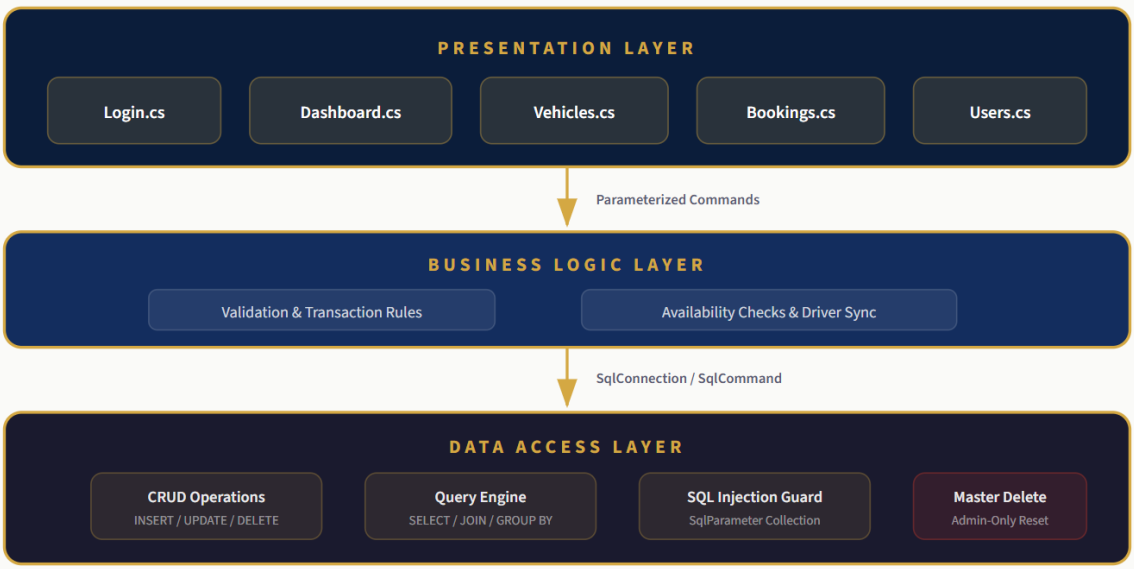

FIGURE 1 — THREE-TIER LOGICAL ARCHITECTURE

3 Database Design and Schema

The backend is powered by a relational database consisting of five primary tables. The schema is designed to prevent data redundancy and maintain referential integrity.

3.1 Entity-Relationship Overview

- **UserTbl:** Manages authentication credentials.
- **CustomerTbl:** A master table for client demographics.
- **VehicleTbl:** Tracks primary assets (Fleet) with logical status flags.
- **DriverTbl:** Records personnel data and performance ratings.
- **BookingTbl:** The central transaction table joining Customers, Vehicles, and Drivers.

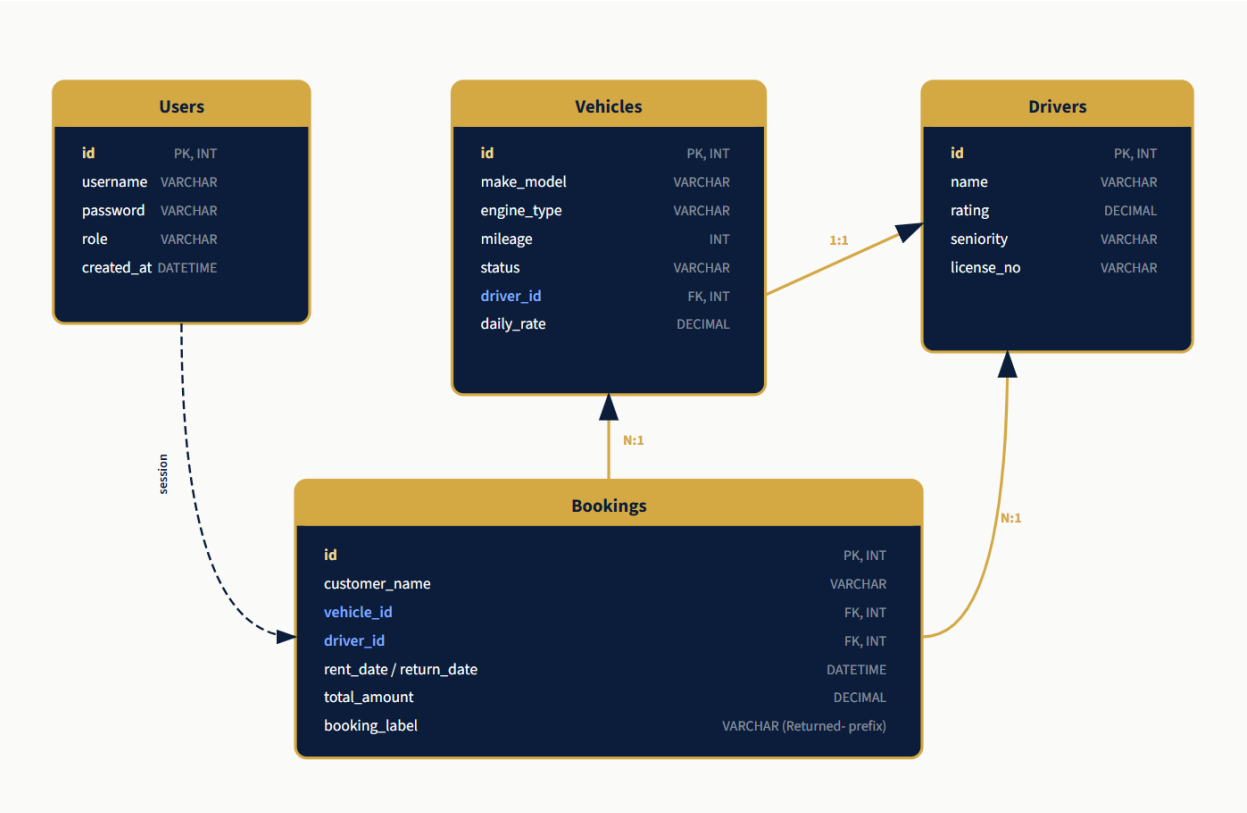

FIGURE 2 — ENTITY-RELATIONSHIP DIAGRAM

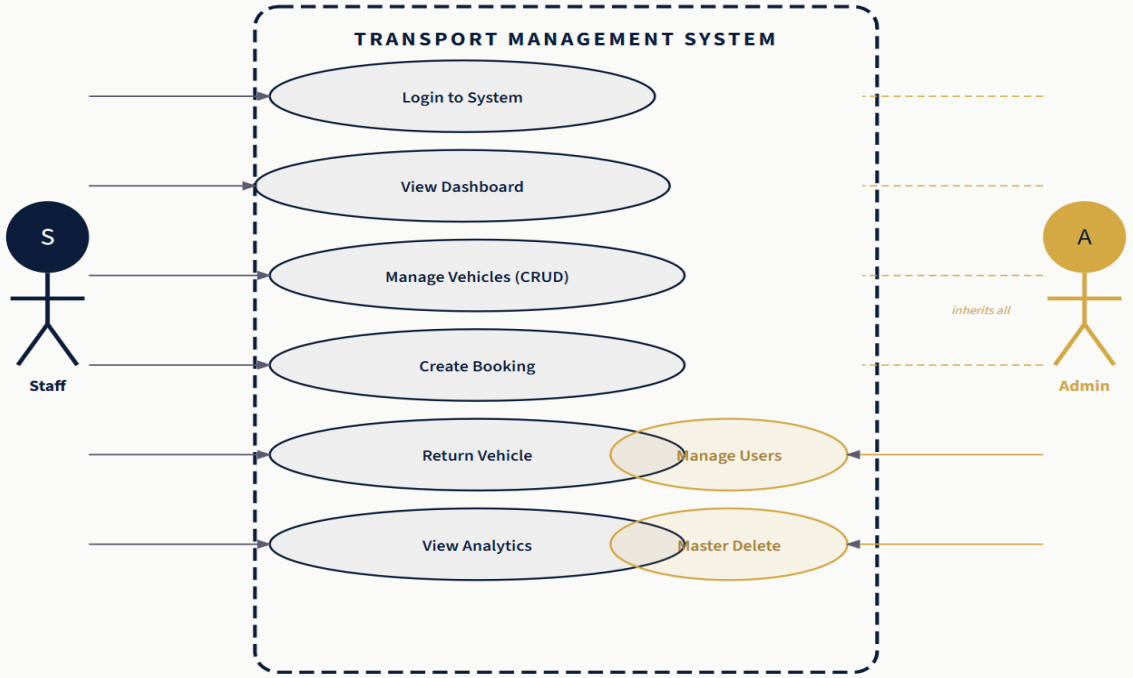

FIGURE 3 — USE CASE DIAGRAM

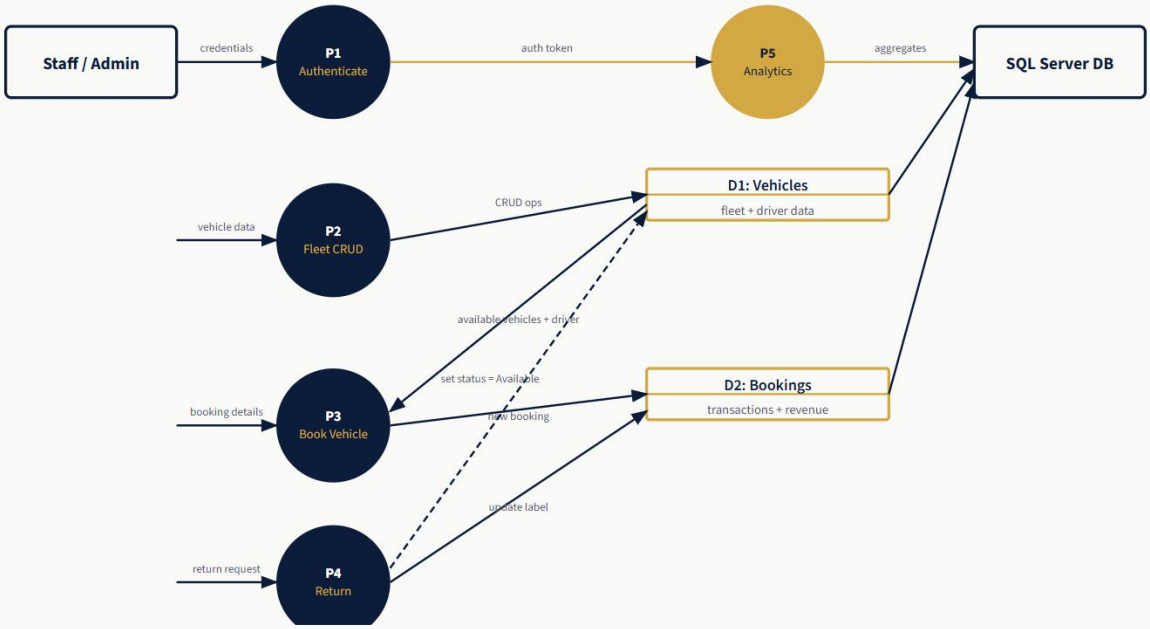

FIGURE 4 — DATA FLOW DIAGRAM (LEVEL 1)

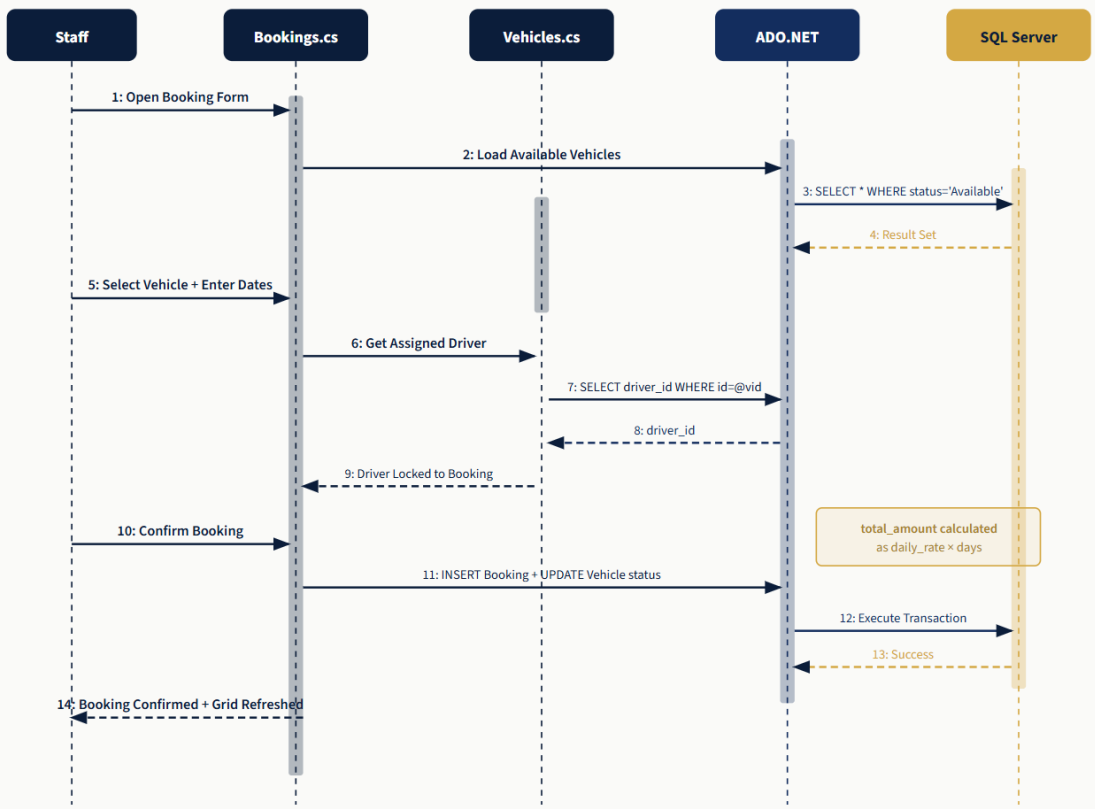

FIGURE 5 — SEQUENCE DIAGRAM: BOOKING CREATION FLOW

4 Object-Oriented Programming (OOP) Application

The TMS is a modular system built on the four pillars of OOP:

4.1 Encapsulation

Variables like the database connection string and primary keys (int Key) are hidden from external forms. This prevents accidental modification of the state during runtime.

4.2 Inheritance and Specialization

By inheriting from System.Windows.Forms.Form, each module specializes in a specific business domain. The use of partial class allows for the separation of auto-generated designer code from developer-written business logic.

5 Containerization: Docker on Windows

To ensure the Transport Management System is portable and easily deployable, the application and its SQL database were containerized using **Docker Desktop**.

5.1 The Dockerfile Process

A Dockerfile was authored to define the environment for the Windows-based application.

Listing 1: Dockerfile for TMS

```
FROM mcr.microsoft.com/dotnet/framework/runtime:4.8windowsservercore-
ltsc2022 WORKDIR /app
COPY ./bin/Release/ .
ENTRYPOINT ["TransportManagementSystem.exe"]
```

5.2 General Deployment Commands

1. **Build the Image:**

```
docker build -t tms-app:latest .
```

2. **Run SQL Container:**

```
docker run -e "ACCEPT_EULA=Y" -e "SA_PASSWORD=
    YourPassword" -p 1433:1433 -d mcr.microsoft.com/mssql/ server:2022-latest
```

6 Security and Data Integrity

- **Parameterized Queries:** To prevent SQL Injection attacks, all database commands utilize cmd.Parameters.AddWithValue.

- **Exception Handling:** Every database operation is wrapped in try-catch-finally blocks to ensure that the application does not crash and the connection is always closed.

7 Conclusion

The Transport Management System is a robust, scalable, and secure desktop application. It successfully demonstrates the power of C# and SQL in solving real-world logistics problems. The project serves as a comprehensive foundation for future enhancements, such as cloud synchronization and automated reporting.